

LIDLbot Documentation

1	Environment Setup	2
1.1	Hardware Prerequisites	2
1.2	Software Prerequisites	2
1.3	Initialize Back-end Environment	2
2	Configuration	3
2.1	Experiment Configuration	4
2.2	Set and Sequence Configuration	4
2.3	Module Configuration	5
2.3.1	Static Modules	5
2.3.2	Block Modules	8
2.3.3	Gating Modules	11
2.4	Block Configuration	12
2.5	Trial Configuration	12
3	Startup	14
4	Session Setup	15
4.1	Participant Stats	15
4.2	Participant Info	15
4.3	App Settings	16
4.4	Launch	16

1 Environment Setup

The following section describes the necessary steps to set up the program source code.

1.1 Hardware Prerequisites

- Two independent computers, minimum 4 GB of RAM, and 500 MB of free storage.

1.2 Software Prerequisites

- Windows or MacOS
- Google Chrome
- Python (3.11 or later)
- Python Poetry (<https://python-poetry.org/docs/>)

1.3 Initialize Back-end Environment

1. Open a terminal in the project root folder.
2. Enter the commands:

```
cd backend
poetry env activate
poetry install
```

Note that `poetry install` is only required for the first time (to install dependencies).

2 Configuration

In LIDLbot, experiments are configured via an experiment folder. On program launch, Researchers are prompted to select a folder on their device, the contents of which define the experiment's flow and stimuli. For general use, no modifications to the source code are required.

The expected structure of the configuration folder is described below.

```
experiment_root/
├── participant_data/
│   ├── responses/
│   │   ├── voice/
│   │   └── chat/
│   ├── data.csv
│   └── participant.csv
├── resources/
│   ├── assets/
│   └── configuration/
│       ├── experiment_config.json
│       ├── sets_sequences_config.json
│       ├── modules_config.json
│       ├── blocks_config.json
│       └── trials_config/
```

Name	Description
experiment_root	This is the root of the configuration folder. There are no restrictions on the folder name.
participant_data	Storage for experiment results. If not present, this folder will be created automatically (along with its contents).
voice	Storage for Participant responses to Voicebot Trials (audio recordings).
chat	Storage for Participant responses to Chatbot Trials (text files).
data.csv	A record of Trial orders and responses chosen by the system (main or alternative), on a per Participant basis.
participant.csv	A record of each Participant's startup settings (Participant ID, set, chosen screen, etc.) as well as start and end timestamps.
resources	Storage for experiment resources.
assets	Storage for images and audio files. Although images and audio files may be placed anywhere within the configuration folder, a dedicated assets folder is recommended.
configuration	Storage for configuration files.
experiment_config.json	General experiment configuration.
sets_sequences_config.json	Definition of Sets and Sequences.
modules_config.json	Definition of Modules.
blocks_config.json	Definition of Blocks.
trials_config	A folder containing (potentially multiple) Trial configuration files.

LIDLbot provides validation for the selected configuration folder and its files. It will ensure that the required files are present and that the configuration files follow the structure as described below. Detected errors will be logged in the program terminal output.

2.1 Experiment Configuration

In `experiment_config.json`, the experiment name is defined. For future iterations of the program, additional settings may also be described here.

A sample entry is provided:

```
{ "experiment_name": "Sample Experiment" }
```

2.2 Set and Sequence Configuration

Sets are defined in `sets_sequences_config.json`. Each **Set** defines a **Sequence** for the assistant screen and another Sequence for the study screen.

A **Sequence** is an ordered and non-empty list of custom **Modules** (see 2.3) that determines the flow of the experiment.

A sample entry is provided:

```
{
  "sample_set_1": {
    "assistant": [
      "sample_session_code_entry",
      "sample_voicebot_block_1"
    ],
    "study": [
      "sample_web_link",
      "sample_content_viewer",
      "sample_session_code_display",
      "sample_prompter_block_1",
      "sample_web_content_embed"
    ]
  },
}
```

2.3 Module Configuration

Modules are defined in `modules_config.json`, (Modules are described in Section 2.2 of the paper). Each Module must specify a valid template.

Module Type	Module Name	Template Label	Description
Static Modules	Content Viewer	<code>content_viewer</code>	Displays content such as images or text. Content will be displayed one at a time, and advanced with a <code>NEXT</code> button. Content displays sequentially in list order.
	Web Survey	<code>web_link</code>	Redirects to an external web page, intended to be used for survey hosting sites such as Qualtrics, Gorilla, or SurveyMonkey. Module will be advanced automatically after detecting the survey has been completed using page redirect logic (must also be configured on the survey website).
	Web Embeds	<code>web_content_embed</code>	Displays embedded content from external web sources in a smaller window, such as videos, widgets, and/or games. Content can be advanced with a <code>NEXT</code> button or an automatic timer.
Gating Modules	Session Code Entry	<code>session_code_entry</code>	Gates Participant progression in the study by requiring a text string (Researcher-specified) acting as a password to proceed.
	Session Code Display	<code>session_code_display</code>	Displays the text string required to proceed.
Block Modules	Prompter	<code>prompter</code>	Displays text, images, or survey URLs by Trials, defined by the Researcher in a trials configuration file.
	Voicebot	<code>voicebot</code>	Built-in voice agent interface. The CUI accepts Participant voice input, which then triggers a premade audio response as specified by the Researcher in a trials configuration file.
	Chatbot	<code>chatbot</code>	Built-in chatbot interface. The CUI accepts Participant voice input, which then triggers a premade audio response as specified by the Researcher in a Trials configuration file.

2.3.1 Static Modules

Static Modules are different kinds of content displays. Broadly, they all involve specifying filepaths or URLs to be displayed.

2.3.1.1 Content Viewer

A Content Viewer displays a series of text or images, primarily used as instructions for the Participant. The content displays sequentially as listed and the viewer comes equipped with a `NEXT` and `PREVIOUS` button

for Participant navigation.

Content within the list may have a type of image or text:

- For text, the content value is simply the desired text.
- For an image, the content value must be the root-relative path to the image file.

A sample entry is provided:

```
"sample_content_viewer": {
  "template": "content_viewer",
  "contents": [
    { "type": "image", "value": "resources/assets/images/sample_cv/sample_cv_1.png" },
    { "type": "image", "value": "resources/assets/images/sample_cv/sample_cv_2.png" },
    { "type": "text", "value": "sample text instruction." }
  ]
}
```



2.3.1.2 Web Survey

A Web Survey is used to redirect users to an external web page. This can be used for surveys, questionnaires, and consent forms. The survey must be capable of redirecting back to the experiment. To assist with this, the following query parameters will automatically be appended to the provided URL.

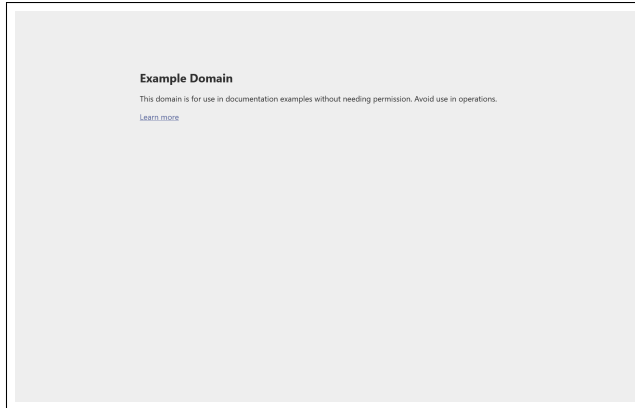
- `redirect_url`: The web survey must return to `redirect_url` after completion.
- `participant_id`: The Participant ID used to identify the study Participant (provided by the Researcher on program startup).

The setup for a Qualtrics-based web survey is described:

1. Add an Embedded Data element (in the Survey Flow) with the parameter name of `redirect_url`. Leave its value blank.
2. Add an End of Survey element.
3. Set its option to Redirect to URL, then set the value to `${e://Field/redirect_url}`.

A sample entry is provided:

```
"sample_web_link": {
  "template": "web_link",
  "url": "https://example.com"
}
```



2.3.1.3 Web Embed

A Web Content Embed displays as an embedded view within the program. This is useful when additional control is needed, such as when enforcing a timer. The `NEXT` button can be initially hidden and set to appear after a specified period of time. The Module can also be set to auto-advance after a specified period of time.

From a configuration standpoint, a `web_content_embed` requires a URL.

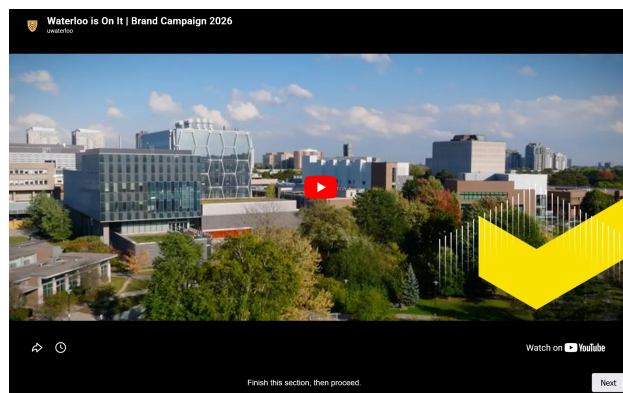
To set the behaviour for the `NEXT` button, specify a timeout period (in seconds) for the field `button_reveal_timeout`. If the value is 0, then the button will immediately be revealed.

Similarly, `auto_advance_timeout` will set a countdown to auto-advance *after* the `NEXT` button appears. If the value is 0, then the Module will immediately advance once the button appears. If the value is `null` or unspecified, then the module will *not* auto-advance.

Finally, Web Content Embed exposes the `embed_style` field, which takes CSS styling to modify the presentation of the embed window.

A sample entry is provided:

```
"sample_web_content_embed": {
  "template": "web_content_embed",
  "url": "https://www.youtube.com/embed/U0_V6A_XXio",
  "button_reveal_timeout": 5,
  "auto_advance_timeout": null,
  "embed_style": "flex: 1;"
}
```



2.3.2 Block Modules

Block Modules are used for customizing the content and stimuli displayed during experimental blocks. Block Modules rely on specific logic to determine the count, selection, and randomization of Trials, which are specified by a Block. Block Modules follow a relationship of Block > Block Modules > Trial.

2.3.2.1 Prompter



Displays text or image prompts. Requires the name of the corresponding Trial Block.

After a Trial Block is completed, a small chime sound will play from the Prompter. This can be used to signal to a nearby Researcher that the Block is finished.

A sample entry is provided:

```
"sample_prompter_block_1": {  
  "template": "prompter",  
  "block": "sample_block_1"  
}
```

2.3.2.2 Voicebot



The Voicebot receives audio input, and replies with audio responses.

The Voicebot has 6 Phases: Standby, Listening, Processing, Response, Reset, and Retry.

The Voicebot starts in the Standby Phase. From here, pressing and holding the SPACE key progresses it to the Listening Phase, where the microphone is active, and the Participant's input is recorded.

The Participant speaks their query, then releases the SPACE key. The number of words spoken is then counted by the Module in the background. By default, the Voicebot Module requires ≥ 3 words spoken to progress. If there are < 3 words, the Voicebot will proceed to the Retry Phase, which displays an "Audio not detected" message, but otherwise behaves as an alternate Standby Phase.

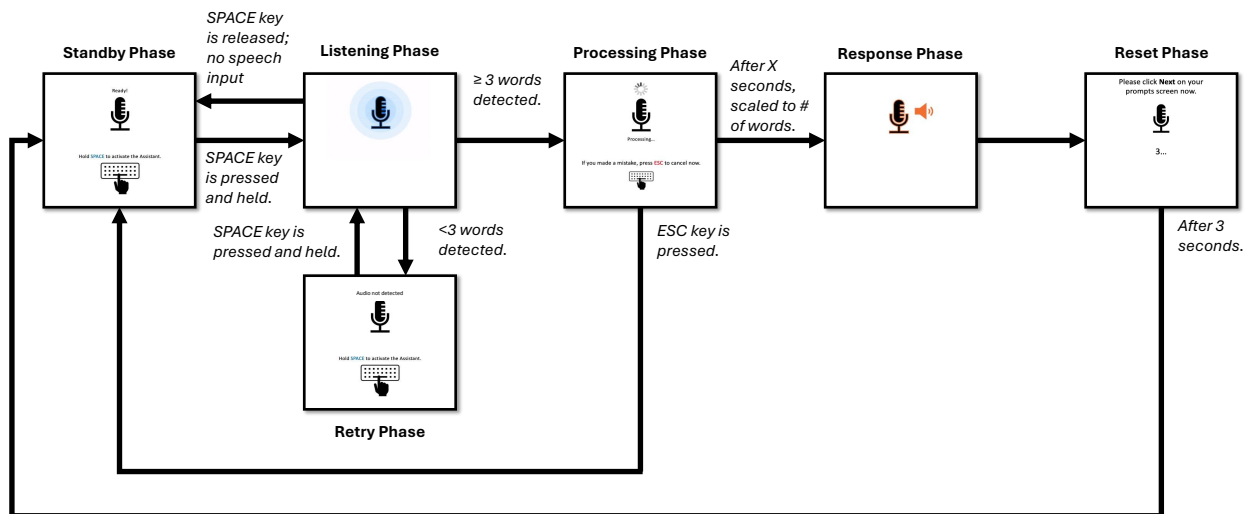
If ≥ 3 words are spoken, the Voicebot will progress to the Processing Phase. Here, a “loading” animation will show. At this Phase, if the ESC key is pressed, the Module will return to the Standby Phase. Otherwise, the Processing Phase will continue until the “processing time” has finished. This processing time is scaled to the number of words spoken. The formula for the processing delay is:

$$Time_{ms} = (3 + N_{words}) * 350$$

(The multiplier is capped at 20 words.)

Then, it will automatically progress to the Response Phase, where the Voicebot will play the predefined response audio for that Trial. At this Phase, the voice input made during the Recording Phase are recorded and saved to the responses/voice folder. (Only inputs that were not cancelled will be saved.)

The Voicebot then proceeds to a Reset Phase, where it will automatically proceed to the Standby Phase after 3 seconds. The displayed text during this Phase reminds the Participant to proceed to the next Trial on the Study Instance.



Configuration requires the name of a Trial Block. The Researcher may optionally change the name of the Voicebot, and the minimum number of words needed to progress.

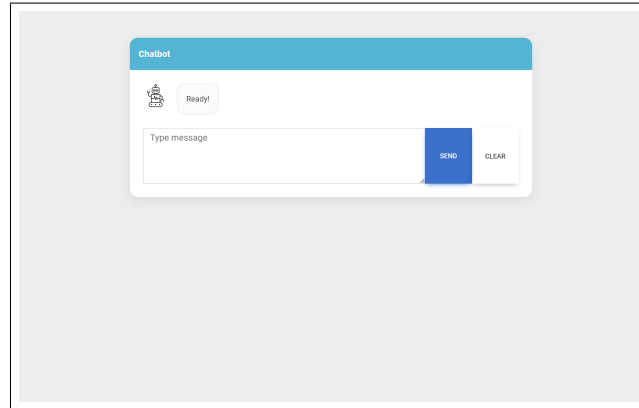
A sample entry is provided:

```

"sample_voice_block_1": {
  "template": "voicebot",
  "block": "sample_block_1",
  "bot_name": "Voicebot",
  "min_words": 3
}

```

2.3.2.3 Chatbot



Chatbot receives text input and replies with text output.

The Chatbot has 3 Phases: Standby, Processing, and Response.

The Chatbot starts in the Standby Phase. From here, the participant enters their text input, and can press the ENTER key or click the “Send” button to progress.

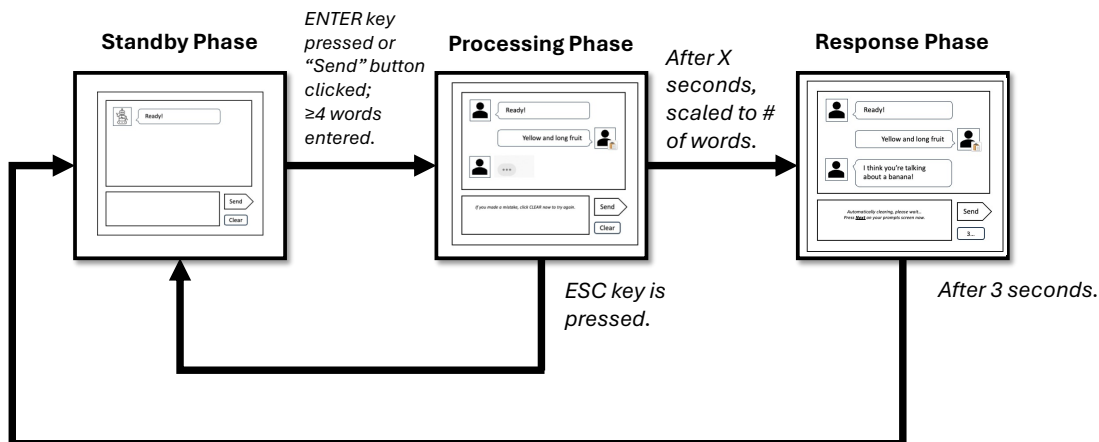
The number of words spoken is then counted by the Module in the background. By default, the Chatbot Module requires ≥ 4 words inputted to progress to the Processing Phase. Here, a “typing” animation will show. At this Phase, if the ESC key is pressed, the Module will return to the Standby Phase. Otherwise, the Processing Phase will continue until the “processing time” has finished. This processing time is scaled to the number of words typed. The formula for the processing delay is:

$$Time_{ms} = (3 + N_{words}) * 400$$

(The multiplier is capped at 13 words.)

Then, it will automatically progress to the Response Phase, where the Chat UI will send the predefined response text for that Trial. At this Phase, the text input entered in the Standby Phase is recorded and saved to the `responses/chat` folder. (Only inputs that were not cancelled will be saved.)

The Chatbot then proceeds to a Reset Phase, where it will automatically proceed to the Standby Phase after 3 seconds. The displayed text during this Phase reminds the Participant to proceed to the next Trial on the Study Instance.



Configuration requires the name of a Trial Block. The Researcher may optionally change the name of the Chatbot, and the minimum number of words needed to progress.

A sample entry is provided:

```
"sample_chat_block_2": {
  "template": "chatbot",
  "block": "sample_block_2",
  "bot_name": "Chatbot",
  "min_words": 4
}
```

2.3.3 Gating Modules

Gating Modules are used to “gate” a Participant’s progression. They prevent a Participant from initiating the Voicebot or Chatbot Modules before having reached the Prompter by requiring a passcode to continue.

2.3.3.1 Session Code Entry

A Session Code Entry prompts the Participant to enter a session code, and will only allow them to proceed if the code is valid (typically used on the Assistant Instance). Configuration requires providing the expected session code.

It is recommended to add a final dummy Session Code Entry screen to the end of an Assistant Instance’s Sequence to maintain immersion.

A sample entry is provided:

```
"sample_session_code_entry": {
  "template": "session_code_entry",
  "code": "sample_code"
}
```

2.3.3.2 Session Code Display

A Session Code Display displays a session code (typically used on the Study Instance). Configuration requires providing the code.

A sample entry is provided:

```
"sample_session_code_display": {
  "template": "session_code_display",
  "code": "sample_code"
}
```

```
}
```



2.4 Block Configuration

In `blocks_config.json`, Trial Blocks are defined. A Block draws its Trials from a Trial configuration file (specified below). The file must be referenced by its root-relative path. A Block also defines the number of Trials to draw from the file, the frequency of alternative responses, and whether the Block should be shuffled.

The `shuffle` field is optional. If it is `true` or unspecified, Trials will be randomly selected from the provided file, then shuffled, and a random subset will be chosen for alternative responses.

If it is `false`, then Trials will be selected from the provided file in the order in which they are presented. They will not be shuffled. However, the subset of Trials chosen for alternative responses will still be randomly selected. If this randomization is not desired, simply set `num_alts` to 0 and define the desired response in the `main_value` of each Trial.

A sample entry is provided:

```
{
  "sample_block_1": {
    "trials": "resources/configuration/trials_config/sample_1_trials_config.json",
    "num_trials_to_sample": 3,
    "num_alts": 2,
    "shuffle": false
  },
  "sample_block_2": {
    "trials": "resources/configuration/trials_config/sample_2_trials_config.json",
    "num_trials_to_sample": 2,
    "num_alts": 1
  }
}
```

2.5 Trial Configuration

Trials are defined in Trial configuration files. Multiple files may be used as a means of organizing Trials into groups. There are no restrictions on the names or locations of these files, however, a dedicated `trial_config` folder, located under `experiment_root/resources/configuration`, is recommended.

Each Trial configuration file must contain a non-empty list of Trials.

Each Trial must define a `prompter` as well as `responses`.

Prompts may have a type of text, image, or survey.

Type	Description
text	The prompt value is simply the desired text.
image	The prompt value must be the root-relative path to the image file.
survey	The prompt value must be a URL to a web survey. The system will redirect to the URL, and the survey must be capable of redirecting back to the experiment. To assist with this, query parameters are automatically appended to the provided URL.
redirect_url	The web survey must return to <code>redirect_url</code> after completion.
participant_id	The Participant ID used to identify the study Participant (provided by the Researcher on program startup).

In a similar manner, responses may have a type of text or audio.

- For text, the main and alternative responses (`main_value` and `alt_value`) are simply the desired texts.
- For audio, `main_value` and `alt_value` must be the root-relative paths to the audio files.

A sample entry is provided:

```
[
  {
    "prompter": {
      "type": "text",
      "value": "Sample prompt text 1."
    },
    "responses": {
      "type": "audio",
      "main_value": "resources/assets/audio/sample_tc/sample_main_audio_1.mp3",
      "alt_value": "resources/assets/audio/sample_tc/sample_alt_audio_1.mp3"
    }
  },
  {
    "prompter": {
      "type": "text",
      "value": "Sample prompt text 2."
    },
    "responses": {
      "type": "audio",
      "main_value": "resources/assets/audio/sample_tc/sample_main_audio_2.mp3",
      "alt_value": "resources/assets/audio/sample_tc/sample_alt_audio_2.mp3"
    }
  }
]
```

3 Startup

1. Open a terminal in the project root folder.
2. Navigate to the backend directory: `cd backend`
3. Run the application: `poetry run python run.py`
4. A file selection dialog will open. Select the experiment configuration folder. A successful validation looks like the output shown below.
 - The contents of the folder (configuration files and assets) will be validated. Validation logs are available in the console.
 - Should validation fail, or should the user fail to select a folder, LIDLbot will terminate.

Starting LIDLbot...

=== Setup ===

Please select the experiment configuration folder in the file dialog window.

Folder selected: C:\---\sample-experiment.

Loading configuration files...

Configuration files loaded successfully.

Validating 'experiment_config.json'...

'experiment_config.json' validated successfully.

Validating 'trial_pools_config.json'...

'trial_pools_config.json' validated successfully.

Validating 'blocks_config.json'...

'blocks_config.json' validated successfully.

Validating 'modules_config.json'...

'modules_config.json' validated successfully.

Validating 'orders_config.json'...

'orders_config.json' validated successfully.

=== Experiment ===

Initializing experiment...

Server running on <http://127.0.0.1:56314>.

Opened browser at <http://127.0.0.1:56314/experiment/page/0>.

6. On successful validation, LIDLbot will automatically launch in the default browser. Ensure that Google Chrome is used; other browsers (e.g., Firefox) may not support the required speech recognition module.
 - (a) The terminal output also provides the URL. If the browser fails to open automatically, or if the default browser is not Google Chrome, copy and paste the URL into a Google Chrome tab.

4 Session Setup

Once launched, LIDLbot begins at the Session Setup screen.

Experiment Settings: Sample Experiment

Participant Stats

Last Used Participant ID

Next Participant ID

1

Total Participant Counts

Participant Info

Participant ID

Enter Participant ID

Secondary ID

Enter Secondary ID

Set

Set1

App Settings

Screen Selection

Assistant

Study

Testing Modes

☐ Dry Run (don't save data)

☐ Bypass ID Restrictions

START

4.1 Participant Stats

Label	Description
Last Used Participant ID	Displays the Participant ID of the most recent previously recorded session.
Next Participant ID	Displays the next Participant ID. By default, the Researcher should enter this value into the Participant ID field unless testing or overwriting a previous entry.
Total Participant Counts	The total number of Participants logged in the experiment.

4.2 Participant Info

Label	Description
Participant ID	Participant ID used to track sessions in records and determine the seed for generating the order of Trials. This ID must match between both Instances, or the Trials will not be synchronized.
Secondary ID	Used for any secondary identification numbers (e.g., SONA).
Set	The Set to be used for the session (see Section 1.2 in the paper).

Information entered in this section is logged to a `participants.csv` file as a record of past sessions. Future development will allow customization of additional Participant fields (e.g., group or condition) to be logged in `participants.csv`.

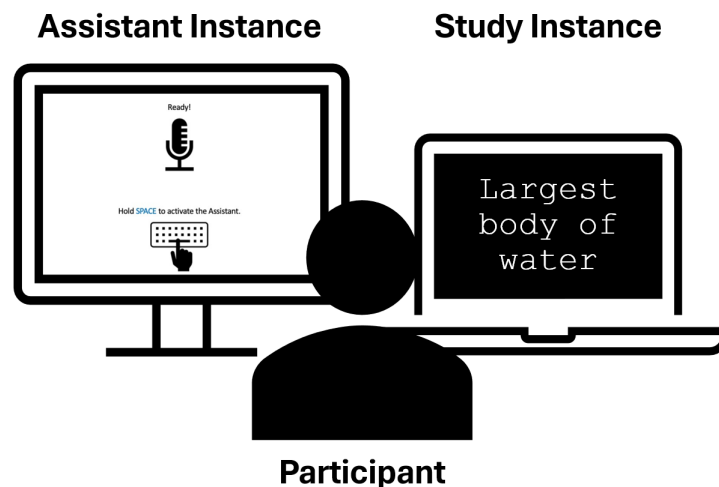
15

4.3 App Settings

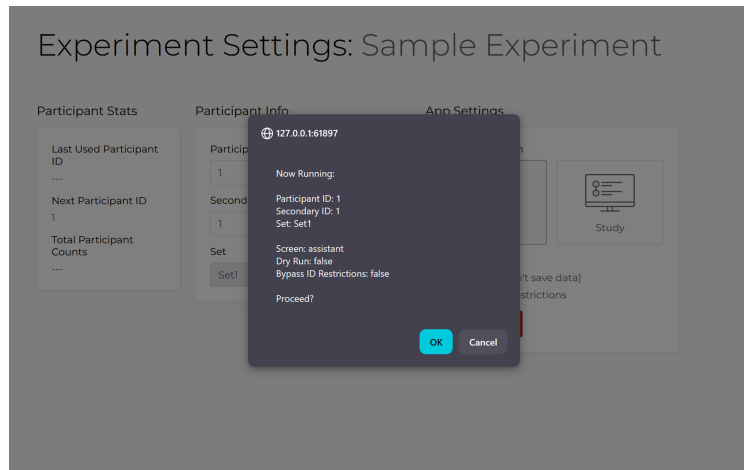
Label	Description
Screen Selection	Selects which Instance is displayed on the device. Typically, one computer displays the Assistant Instance and the other displays the Study Instance.
Testing Modes (Dry Run)	Debugging mode that allows a full session to be run without saving Participant data or input.
Testing Modes (Bypass ID Restrictions)	Overrides the default restriction that prevents launching a session if the Participant ID does not match the Next Participant ID value.

4.4 Launch

Two computers are needed to use the LIDLbot system. It is recommended to have the computers side-by-side, with the Assistant Instance on the left, and the Study Instance on the right.



1. Launch LIDLbot on the two computers.
2. Enter the Participant ID. This number must match the value listed under Next Participant ID in the Participant Stats box.
3. (Optional) If used, enter the Secondary ID.
4. Select the Set for the current experiment session.
5. Select Assistant on your first computer, and then Study on the second.
6. Click START when ready to begin.
7. After clicking Start, an alert dialog will appear that lists the Participant ID and other entered information.
 - The Researcher should verify that entries match between both Instances, particularly the Participant ID and Set, as mismatches may cause desynchronization.
8. Click OK to begin.



For better immersion:

- It is recommended to fullscreen the application to minimize distractions and improve immersion. This can be done by pressing F11 on Windows or clicking the green window button on MacOS.
- It is recommended that the Researcher launches the experiment before Participants enter the testing area for better immersion.